

My Project

Generated by Doxygen 1.13.2

1 Release aprašymai	1
1.1 v.pradinė	1
1.2 v0.1	1
1.3 V0.2	1
1.4 V0.3	1
1.5 V0.3.1	1
1.6 V0.4	1
1.7 V0.4.1	2
1.8 V0.4.2	2
1.9 V1.0 pradinis relizas	2
1.10 Naudojimos instrukcija	2
1.11 Diegimo instrukcija	2
1.12 Tyrimai	3
1.12.0.1 Matavimo sistemos parametrai:	3
1.12.1 Pirmas tyrimas (Failų kūrimas ir jų uždarymas)	3
1.12.2 Antrasis tyrimas (Duomenų apdorojimas)	3
1.12.2.1 1,000 studentų	3
1.12.2.2 10,000 studentų	3
1.12.2.3 100,000 studentų	4
1.12.2.4 1,000,000 studentų	4
1.12.2.5 10,000,000 studentų	4
1.12.3 Trečias tyrimas (Konteinerių testavimas)	4
1.12.4 Naudojant <code>std::vector</code>	5
1.12.4.1 1,000 studentų	5
1.12.4.2 10,000 studentų	5
1.12.4.3 100,000 studentų	5
1.12.4.4 1,000,000 studentų	5
1.12.4.5 10,000,000 studentų	5
1.12.5 Naudojant <code>std::list</code>	5
1.12.5.1 1,000 studentų	5
1.12.5.2 10,000 studentų	5
1.12.5.3 100,000 studentų	6
1.12.5.4 1,000,000 studentų	6
1.12.5.5 10,000,000 studentų	6
1.12.6 Naudojant <code>std::deque</code>	6
1.12.6.1 1,000 studentų	6
1.12.6.2 10,000 studentų	6
1.12.6.3 100,000 studentų	6
1.12.6.4 1,000,000 studentų	7
1.12.6.5 10,000,000 studentų	7
1.12.7 Ketvirtas tyrimas (Skirstymas)	7
1.12.7.1 Vector	7

1.12.7.2 1 Strategija	7
1.12.7.3 2 Strategija	7
1.12.7.4 3 Strategija	8
1.12.7.5 List	8
1.12.7.6 1 Strategija	8
1.12.7.7 2 Strategija	8
1.12.7.8 3 Strategija	8
1.12.7.9 Deque	8
1.12.7.10 1 Strategija	8
1.12.7.11 2 Strategija	9
1.12.7.12 3 Strategija	9
1.12.8 Penktas tyrimas (Struct vs Class)	9
1.12.8.1 Veikimo greitis -O1	9
1.12.8.2 Veikimo greitis -O2	10
1.12.8.3 Veikimo greitis -O3	10
2 Hierarchical Index	11
2.1 Class Hierarchy	11
3 Class Index	13
3.1 Class List	13
4 File Index	15
4.1 File List	15
5 Class Documentation	17
5.1 Calc_Students Class Reference	17
5.2 CL_Students Class Reference	17
5.3 CLInputs Class Reference	18
5.4 File_students Class Reference	18
5.5 FileGen Class Reference	18
5.6 Generator Class Reference	19
5.7 Output_students Class Reference	19
5.8 Person Class Reference	19
5.9 Process_settings Struct Reference	20
5.10 Stopwatch Class Reference	20
5.11 Student Class Reference	21
5.11.1 Member Function Documentation	22
5.11.1.1 get_full_name()	22
5.12 StudentTest Class Reference	22
5.13 Timer Class Reference	22
6 File Documentation	23
6.1 cl_students.h	23

6.2 constant_definitions.h	23
6.3 file_gen.h	23
6.4 generator.h	24
6.5 inputs.h	24
6.6 main.h	24
6.7 output_students.h	25
6.8 person.h	25
6.9 process_settings.h	26
6.10 read_students.h	26
6.11 student.h	26
6.12 student_test.h	27
6.13 students_calc.h	27
6.14 timer.h	28
Index	29

Chapter 1

Release aprašymai

1.1 v.pradinė

Įvadiniai studentų duomenys ir išvedami jų apskaičiuoti vidurkiai ar medianos į komdų eilutę

1.2 v0.1

Galima generuoti studentų duomenis ir sukurtos 2 projekto versijos 1 - naudojant `std::vector`, 2 - naudojant `c` masyvus

1.3 V0.2

Pridėtas duomenų įvesties į ir išvesties iš failo pasirinkimas, panaikinta `c` masyvą naudojanti programos versija

1.4 V0.3

Pridėtas duomenų rūšiavimas pagal pasirinktus kriterijus

1.5 V0.3.1

Klaidų žinutės lietuvių kalba

1.6 V0.4

Sukurtas `README.md` failas kuriame surašyti programos spartos matmenys ir pridėtas studentų failo generavimo funkcionalumas

1.7 V0.4.1

Pataisyti ir papildyti matavimai, klaidų ištaisymas

1.8 V0.4.2

Klaidos pataisymas

1.9 V1.0 pradinis relizas

Sukurtos papildomai dvi programos veikimo versijos, papildytas README programos veikimo spartos matmenimis ir testavimo detalėmis

1.10 Naudojimos instrukcija

- Paleidimas: Nueiti į norimo naudojimo konteinerio build aplanką ir paleisti programą "1-oji-uzduotis"
- Skaičiavimų atlikimas:
 1. Paleidus programą duodamas pasirinkimas 1-6, jeigu norite atlikti skaičiavimus su studentai pasirinkite 1-4 pagal poreikį (ką pasirinkimai daro bus aprašyta programos veikimo metu)
 2. Jeigu pasirinkote 4 pasirinkimą įveskite studentų failo pavadinimą (studentų failas turi būti tame pačiame aplankale iš kurios ir paleidote programą)
 3. Pasirinkite rūšiavimo būdą (vidurkių ir medianų rūšiavimas yra nuo mažiausio iki didžiausio)
 4. Pasirinkite išvesties būdą
 5. Pasirinkite skirstymo strategiją
 6. Jeigu pasirinkote įrašyti pažimius ranka ar generuoti juos sekite prgramoje nurodytas instrukcijas

1.11 Diegimo instrukcija

1. Nueikite į norimos programos versijos aplankalo (Deque/List/Vector)
 - `cd Vector`
Arba
 - `cd List`
Arba
 - `cd Deque`
2. Sukurkite build aplanką ir įeikite į jį
`mkdir build && cd build`
3. Paleiskite cmake comandą
`cmake ..`
4. Išeikite iš aplankalo ir sukompiliuokite programą
`cd .. && cmake --build build`
5. Programa bus build aplankale su pavadinimu "1-oji-uzduotis"

1.12 Tyrimai

1.12.0.1 Matavimo sistemos parametrai:

CPU: 7800x3d \ **RAM:** 32GB | 6000 MT/s \ **SSD:** NVMe M.2 | 7,000/7,000MB/s read/write

1.12.1 Pirmas tyrimas (Failų kūrimas ir jų uždarymas)

- Kiekvienas studentas turi po 10 namų darbų rezultatų + egzaminas
- Laiko matavimui naudojam `chrono` biblioteka
- Matavimai atliekami penkis kartus ir iš jų išgaunami vidurkiai
- Kodas buvo sukompiliuotas naudojant `-O3` gaire

Studentu sk.	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
1,000	866 s	966 s	877 s	792 s	936 s	887 s
10,000	6486 s	6903 s	6497 s	6364 s	6307 s	6511 s
100,000	68 ms	66 ms	65 ms	66 ms	67 ms	66 ms
1,000,000	594 ms	592 ms	587 ms	595 ms	608 ms	595 ms
10,000,000	5917 ms	5827 ms	5871 ms	5843 ms	5887 ms	5869 ms

>Failo kūrimo pavyzdys:

1.12.2 Antrasis tyrimas (Duomenų apdorojimas)

- Laiko matavimui naudojam `chrono` biblioteka
- Duomenys gaunami iš ankščiau sugeneruotų failų
- Kiekvienas studentas turi po 10 namų darbų rezultatų + egzaminas
- Matavimai atliekami penkis kartus ir iš jų išgaunami vidurkiai
- Matuojami vykdomi su penkiais skirtingais failo dydžiais
- Kodas buvo sukompiliuotas naudojant `-O3` gaire
- Matuojami:
 - nuskaitymas iš failo
 - studentų rūšiavimą į dvi grupes/kategorijas
 - surūšiuotų studentų išvedimą į du naujus failus
 - visos programos veikimo laikas

1.12.2.1 1,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	545 s	639 s	518 s	509 s	485 s	539 s
Rūšiavimas	101 s	134 s	100 s	101 s	101 s	107 s
Išvedimas	340 s	435 s	394 s	362 s	369 s	380 s
Veikimo laikas	1092 s	1334 s	1113 s	1076 s	1056 s	1134 s

1.12.2.2 10,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	5524 s	5118 s	5430 s	5809 s	5024 s	5381 s
Rūšiavimas	1043 s	1129 s	1133 s	1107 s	1106 s	1104 s
Išvedimas	2795 s	2835 s	2870 s	2859 s	2770 s	2826 s
Veikimo laikas	10321 s	10053 s	10386 s	10760 s	9873 s	10279 s

1.12.2.3 100,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	47 ms	48 ms	46 ms	49 ms	47 ms	47 ms
Rūšiavimas	7 ms	7 ms	6 ms	6 ms	6 ms	6 ms
Išvedimas	27 ms	27 ms	25 ms	32 ms	33 ms	29 ms
Veikimo laikas	90 ms	92 ms	88 ms	98 ms	97 ms	93 ms

1.12.2.4 1,000,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	440 ms	443 ms	443 ms	444 ms	444 ms	443 ms
Rūšiavimas	51 ms	51 ms	53 ms	50 ms	52 ms	51 ms
Išvedimas	270 ms	273 ms	275 ms	267 ms	268 ms	270 ms
Veikimo laikas	848 ms	855 ms	859 ms	849 ms	853 ms	853 ms

1.12.2.5 10,000,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	4476 ms	4483 ms	4502 ms	4465 ms	4491 ms	4483 ms
Rūšiavimas	668 ms	677 ms	685 ms	670 ms	679 ms	676 ms
Išvedimas	2602 ms	2628 ms	2586 ms	2640 ms	2665 ms	2624 ms
Veikimo laikas	8686 ms	8750 ms	8730 ms	8727 ms	8781 ms	8735 ms

1.12.3 Trečias tyrimas (Konteinerių testavimas)

- Laiko matavimui naudojam `chrono` biblioteka
- Duomenys gaunami iš ankščiau sugeneruotų failų
- Kiekvienas studentas turi po 10 namų darbų rezultatų + egzaminas
- Matavimai atliekami penkis kartus ir iš jų išgaunami vidurkiai
- Matuojami vykdomi su penkiais skirtingais failo dydžiais
- Kodas buvo sukompiliuotas naudojant `-O3` gaire
- Duomenys rūšiuojami studento pažymių vidurkio didėjimo tvarka
- Matuojas:
 - Duomenų nuskaitymas iš failų į atitinkamą konteinerį
 - Studentų rūšiavimas didėjimo tvarką konteineryje
 - Studentų skirstymas į dvi grupes/kategorijas
- Studentams saugoti bus naudojami 3 skirtingi konteineriai:
 - `std::vector`
 - `std::list`
 - `std::deque`

1.12.4 Naudojant `std::vector`

1.12.4.1 1,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	504 s	573 s	582 s	609 s	495 s	552 s
Rušiavimas	53 s	65 s	67 s	64 s	52 s	60 s
Skirstymas	98 s	119 s	112 s	122 s	99 s	110 s

1.12.4.2 10,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	5396 s	5124 s	5232 s	4846 s	5028 s	5125 s
Rušiavimas	826 s	745 s	785 s	939 s	943 s	847 s
Skirstymas	955 s	941 s	965 s	997 s	1099 s	991 s

1.12.4.3 100,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	49040 s	46073 s	48257 s	49288 s	50360 s	48603 s
Rušiavimas	11722 s	12060 s	11927 s	12071 s	12136 s	11983 s
Skirstymas	9479 s	9184 s	8919 s	9446 s	9274 s	9260 s

1.12.4.4 1,000,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	430 ms	435 ms	437 ms	443 ms	436 ms	436 ms
Rušiavimas	134 ms	140 ms	145 ms	141 ms	136 ms	139 ms
Skirstymas	111 ms	109 ms	110 ms	110 ms	109 ms	109 ms

1.12.4.5 10,000,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	4414 ms	4401 ms	4396 ms	4444 ms	4414 ms	4413 ms
Rušiavimas	1743 ms	1665 ms	1729 ms	1722 ms	1660 ms	1703 ms
Skirstymas	1350 ms	1357 ms	1348 ms	1343 ms	1350 ms	1349 ms

1.12.5 Naudojant `std::list`

1.12.5.1 1,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	560 s	576 s	606 s	485 s	594 s	564 s
Rušiavimas	52 s	64 s	66 s	52 s	65 s	59 s
Skirstymas	99 s	121 s	119 s	94 s	126 s	111 s

1.12.5.2 10,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	4843 s	5330 s	4923 s	4755 s	5391 s	5048 s
Rušiavimas	761 s	738 s	756 s	757 s	735 s	749 s
Skirstymas	1005 s	1068 s	894 s	882 s	963 s	962 s

1.12.5.3 100,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	47977 s	47000 s	48757 s	49032 s	59094 s	50372 s
Rušiavimas	12449 s	12087 s	12294 s	12273 s	12074 s	12235 s
Skirstymas	9721 s	9184 s	10342 s	9746 s	9834 s	9765 s

1.12.5.4 1,000,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	476 ms	472 ms	477 ms	477 ms	473 ms	475 ms
Rušiavimas	285 ms	233 ms	256 ms	243 ms	233 ms	250 ms
Skirstymas	167 ms	155 ms	168 ms	165 ms	162 ms	163 ms

1.12.5.5 10,000,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	4643 ms	4681 ms	4723 ms	4707 ms	4613 ms	4673 ms
Rušiavimas	5225 ms	5597 ms	5502 ms	5185 ms	5349 ms	5371 ms
Skirstymas	1647 ms	1710 ms	1669 ms	1654 ms	1658 ms	1667 ms

1.12.6 Naudojant std::deque

1.12.6.1 1,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	589 s	511 s	480 s	565 s	593 s	547 s
Rušiavimas	133 s	108 s	100 s	124 s	126 s	118 s
Skirstymas	102 s	84 s	81 s	99 s	91 s	91 s

1.12.6.2 10,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	4974 s	4865 s	4524 s	4909 s	4793 s	4813 s
Rušiavimas	1103 s	1213 s	1129 s	1070 s	1106 s	1124 s
Skirstymas	891 s	814 s	760 s	852 s	768 s	817 s

1.12.6.3 100,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	45226 s	45315 s	57134 s	46732 s	45353 s	47952 s
Rušiavimas	13310 s	13076 s	13947 s	12625 s	12929 s	13177 s
Skirstymas	8509 s	8437 s	8048 s	8168 s	8224 s	8277 s

1.12.6.4 1,000,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	453 ms	461 ms	458 ms	452 ms	453 ms	455 ms
Rušiavimas	159 ms	166 ms	155 ms	164 ms	164 ms	161 ms
Skirstymas	128 ms	132 ms	140 ms	136 ms	127 ms	132 ms

1.12.6.5 10,000,000 studentų

Matavimas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
Nuskaitymas	4520 ms	4489 ms	4540 ms	4530 ms	5122 ms	4640 ms
Rušiavimas	2040 ms	1952 ms	1952 ms	1982 ms	1969 ms	1979 ms
Skirstymas	1496 ms	1479 ms	1503 ms	1482 ms	1500 ms	1492 ms

1.12.7 Ketvirtas tyrimas (Skirstymas)

- Laiko matavimui naudojam `chrono` biblioteka
- Duomenys gaunami iš ankščiau sugeneruotų failų
- Kiekvienas studentas turi po 10 namų darbų rezultatų + egzaminas
- Matavimai atliekami penkis kartus ir iš jų išgaunami vidurkiai
- Kodas buvo sukompiliuotas naudojant `-O3` gaire
- Matuojami vykdomi su penkiais skirtingais failo dydžiais
- Duomenys nerūšiuojami
- Naudojamos 3 strategijos:
 1. Bendro studentai konteinerio skaidymas (rūšiavimas) į du naujus to paties tipo konteinerius
 2. Bendro studentų konteinerio skaidymas (rūšiavimas) panaudojant tik vieną naują konteinerį
 3. Bendro studentų konteinerio skaidymas (rūšiavimas) panaudojant "efektyvius" darbo su konteineriais metodus (`erase`, `remove_if`)

1.12.7.1 Vector

1.12.7.2 1 Strategija

Studentu sk.	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
1,000	86 s	82 s	103 s	87 s	80 s	87 s
10,000	1064 s	1212 s	1122 s	1155 s	1119 s	1134 s
100,000	7906 s	7346 s	7287 s	7355 s	7335 s	7445 s
1,000,000	52 ms	53 ms	53 ms	49 ms	53 ms	52 ms
10,000,000	711 ms	707 ms	701 ms	699 ms	707 ms	705 ms

1.12.7.3 2 Strategija

Studentu sk.	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
1,000	58 s	70 s	71 s	60 s	69 s	65 s
10,000	807 s	761 s	796 s	769 s	737 s	774 s
100,000	6122 s	5950 s	6083 s	6338 s	5944 s	6087 s
1,000,000	46 ms	47 ms	46 ms	46 ms	47 ms	46 ms
10,000,000	571 ms	570 ms	560 ms	562 ms	564 ms	565 ms

1.12.7.4 3 Strategija

Studentu sk.	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
1,000	36 s	41 s	48 s	43 s	40 s	41 s
10,000	616 s	562 s	571 s	614 s	614 s	595 s
100,000	4613 s	4101 s	4130 s	4014 s	3839 s	4139 s
1,000,000	28 ms	28 ms	29 ms	29 ms	33 ms	29 ms
10,000,000	355 ms	356 ms	354 ms	355 ms	351 ms	354 ms

1.12.7.5 List

1.12.7.6 1 Strategija

Studentu sk.	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
1,000	138 s	103 s	97 s	103 s	119 s	112 s
10,000	1207 s	1020 s	1117 s	953 s	994 s	1058 s
100,000	9856 s	9813 s	9280 s	9690 s	10011 s	9730 s
1,000,000	96 ms	100 ms	97 ms	97 ms	98 ms	97 ms
10,000,000	946 ms	956 ms	956 ms	948 ms	1000 ms	961 ms

1.12.7.7 2 Strategija

Studentu sk.	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
1,000	17 s	21 s	18 s	25 s	16 s	19 s
10,000	218 s	177 s	175 s	181 s	179 s	186 s
100,000	1699 s	1693 s	1693 s	1682 s	1678 s	1689 s
1,000,000	16 ms	17 ms	17 ms	17 ms	16 ms	16 ms
10,000,000	172 ms	172 ms	173 ms	170 ms	170 ms	171 ms

1.12.7.8 3 Strategija

Studentu sk.	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
1,000	60 s	50 s	63 s	49 s	50 s	54 s
10,000	529 s	517 s	674 s	553 s	677 s	590 s
100,000	5377 s	5615 s	5088 s	5006 s	5225 s	5262 s
1,000,000	66 ms	69 ms	68 ms	68 ms	66 ms	67 ms
10,000,000	668 ms	668 ms	685 ms	665 ms	676 ms	672 ms

1.12.7.9 Deque

1.12.7.10 1 Strategija

Studentu sk.	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
1,000	84 s	85 s	138 s	107 s	84 s	99 s
10,000	844 s	804 s	786 s	898 s	832 s	832 s
100,000	10858 s	8951 s	8304 s	7947 s	8524 s	8916 s
1,000,000	82 ms	84 ms	80 ms	80 ms	80 ms	81 ms
10,000,000	820 ms	818 ms	810 ms	820 ms	819 ms	817 ms

1.12.7.11 2 Strategija

Studentu sk.	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
1,000	37 s	34 s	33 s	35 s	29 s	33 s
10,000	248 s	249 s	250 s	266 s	257 s	254 s
100,000	2483 s	2414 s	2526 s	2483 s	2520 s	2485 s
1,000,000	24 ms	24 ms	24 ms	24 ms	23 ms	23 ms
10,000,000	243 ms	253 ms	239 ms	243 ms	243 ms	244 ms

1.12.7.12 3 Strategija

Studentu sk.	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
1,000	47 s	48 s	53 s	57 s	58 s	52 s
10,000	460 s	446 s	452 s	425 s	436 s	443 s
100,000	4231 s	4184 s	4566 s	4238 s	4094 s	4262 s
1,000,000	49 ms	46 ms	48 ms	49 ms	48 ms	48 ms
10,000,000	465 ms	466 ms	467 ms	465 ms	468 ms	466 ms

1.12.8 Penktas tyrimas (Struct vs Class)

- Laiko matavimui naudojam `chrono` biblioteka
- Duomenys gaunami iš ankščiau sugeneruotų failų
- Kiekvienas studentas turi po 10 namų darbų rezultatų + egzaminas
- Matavimai atliekami penkis kartus ir iš jų išgaunami vidurkiai
- Matuojami du kodo atvejai, Vienas naudojant Struktūros studentams kitas klases
- Matavimui naudojami tik 1,000,000 ir 10,000,000 studentų kiekiai
- Matavimui bus lyginami -O1, -O2 ir -O3 kompiliavimo gairės (3 atvejai, kodo veikimo greitis ir bin failo dydis)
- Matuojamas pilnas programos veikimo laikas (neskaitant vartotojo įvesties)
- Naudojamas vector tipo konteineris studentų masyui saugoti

1.12.8.1 Veikimo greitis -O1

(stud. sk.) Duom. tipas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
(1,000,000) Klasė	1562 ms	1619 ms	1616 ms	1630 ms	1593 ms	1604 ms
(1,000,000) Struktūra	1306 ms	1333 ms	1321 ms	1328 ms	1311 ms	1319 ms
(10,000,000) Klasė	17244 ms	17597 ms	17800 ms	17520 ms	17590 ms	17550 ms
(10,000,000) Struktūra	13950 ms	14018 ms	14012 ms	14125 ms	14019 ms	14024 ms

Klasės metodo bin dydis: 107.94 KiB Struktūros metodo bin dydis: 110.88 KiB

1.12.8.2 Veikimo greitis -O2

(stud. sk.) Duom. tipas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
(1,000,000) Klasė	1559 ms	1607 ms	1599 ms	1606 ms	1597 ms	1593 ms
(1,000,000) Struktūra	1443 ms	1338 ms	1337 ms	1359 ms	1358 ms	1367 ms
(10,000,000) Klasė	16920 ms	17292 ms	17082 ms	17048 ms	17276 ms	17123 ms
(10,000,000) Struktūra	14104 ms	14132 ms	14284 ms	14277 ms	14272 ms	14213 ms

Klasės metodo bin dydis: 117.92 KiB Struktūros metodo bin dydis: 119.91 KiB

1.12.8.3 Veikimo greitis -O3

(stud. sk.) Duom. tipas	Laikas 1	Laikas 2	Laikas 3	Laikas 4	Laikas 5	Vidurkiai
(1,000,000) Klasė	1569 ms	1556 ms	1550 ms	1551 ms	1572 ms	1559 ms
(1,000,000) Struktūra	1264 ms	1329 ms	1291 ms	1278 ms	1292 ms	1290 ms
(10,000,000) Klasė	16396 ms	16528 ms	16528 ms	16639 ms	16589 ms	16536 ms
(10,000,000) Struktūra	13311 ms	13454 ms	13388 ms	13425 ms	13332 ms	13382 ms

Klasės metodo bin dydis: 129.52 KiB Struktūros metodo bin dydis: 123.69 KiB

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Calc_Students	17
CL_Students	17
CLInputs	18
File_students	18
FileGen	18
Generator	19
Output_students	19
Person	19
Student	21
Process_settings	20
Stopwatch	20
StudentTest	22
Timer	22

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Calc_Students	17
CL_Students	17
CLInputs	18
File_students	18
FileGen	18
Generator	19
Output_students	19
Person	19
Process_settings	20
Stopwatch	20
Student	21
StudentTest	22
Timer	22

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

cl_students.h	23
constant_definitions.h	23
file_gen.h	23
generator.h	24
inputs.h	24
main.h	24
output_students.h	25
person.h	25
process_settings.h	26
read_students.h	26
student.h	26
student_test.h	27
students_calc.h	27
timer.h	28

Chapter 5

Class Documentation

5.1 Calc_Students Class Reference

Static Public Member Functions

- static void **calc_grades** (std::vector< [Student](#) > &students)
- static void **set_student_avg** ([Student](#) &student)
- static void **set_student_median** ([Student](#) &student)
- static void **sort_students** (std::vector< [Student](#) > &students, int sort_method)
- static bool **student_sort_f_name** ([Student](#) const &lhs, [Student](#) const &rhs)
- static bool **student_sort_l_name** ([Student](#) const &lhs, [Student](#) const &rhs)
- static bool **student_sort_avg** ([Student](#) const &lhs, [Student](#) const &rhs)
- static bool **student_sort_med** ([Student](#) const &lhs, [Student](#) const &rhs)
- static double **calc_average** (std::vector< int > &&scores, int &&egzam)

The documentation for this class was generated from the following files:

- students_calc.h
- students_calc.cpp

5.2 CL_Students Class Reference

Static Public Member Functions

- static std::vector< [Student](#) > **get_user_input** (const bool &generate_names, const bool &generate_grades)
- static void **gen_names** ([Student](#) &student)
- static void **CL_names** ([Student](#) &student, int students_size)
- static void **gen_grades** ([Student](#) &student, int students_size)
- static void **CL_grades** ([Student](#) &student, int students_size)

The documentation for this class was generated from the following files:

- cl_students.h
- cl_students.cpp

5.3 CLInputs Class Reference

Static Public Member Functions

- static int **numInput** (const std::string &prompt, const int limit_max=int_lim::max(), const int limit_min=int_lim::min())
- static std::string **strInput** (const std::string &prompt, const int limit_max=int_lim::max(), const int limit_min=int_lim::min())

The documentation for this class was generated from the following files:

- inputs.h
- inputs.cpp

5.4 File_students Class Reference

Public Member Functions

- **File_students** (const std::string &file_name)
- void **open** (const std::string &file_name)
- void **openFile** (const std::string &file_name)
- void **closeFile** ()
- void **count_hw** ()
- void **read_students** (std::vector< [Student](#) > &students)

Public Attributes

- int **iteration** = 0
- std::string **error**
- bool **file_opened** = false

The documentation for this class was generated from the following files:

- read_students.h
- read_students.cpp

5.5 FileGen Class Reference

Static Public Member Functions

- static void **gen_file** (std::uint64_t num_students, std::uint64_t num_hw, const std::string &file_name)

The documentation for this class was generated from the following files:

- file_gen.h
- file_gen.cpp

5.6 Generator Class Reference

Public Member Functions

- `std::vector< std::string > gen_names (unsigned long num_of_name) const`

Static Public Member Functions

- `static std::string gen_f_name ()`
- `static std::string gen_l_name ()`
- `static std::string gen_name ()`
- `static int rand_int (const int min, const int max)`

The documentation for this class was generated from the following files:

- `generator.h`
- `generator.cpp`

5.7 Output_students Class Reference

Public Member Functions

- `void open_file (const std::string &file_name)`
- `void close_file ()`
- `void output_students (std::vector< Student > const &students, const bool &out_to_file)`
- `void output_file (std::vector< Student > const &students)`
- `void output_terminal (std::vector< Student > const &students)`

Public Attributes

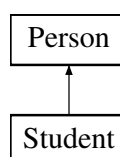
- `std::string error`

The documentation for this class was generated from the following files:

- `output_students.h`
- `output_students.cpp`

5.8 Person Class Reference

Inheritance diagram for Person:



Public Member Functions

- **Person** (std::string f_name, std::string l_name)
- std::string **get_f_name** () const
- std::string **get_l_name** () const
- void **set_f_name** (std::string f_name)
- void **set_l_name** (std::string l_name)
- virtual std::string **get_full_name** ()=0

Protected Attributes

- std::string **f_name**
- std::string **l_name**

The documentation for this class was generated from the following file:

- person.h

5.9 Process_settings Struct Reference

Public Attributes

- bool **generate_names** = false
- bool **generate_grades** = false
- bool **get_students_from_file** = false
- int **sort_method** = 0
- bool **output_to_file** = false
- bool **generate_input_file** = false
- int **input_file_student_num** = 0
- int **input_file_student_hw** = 0
- std::string **input_file_name**
- int **distribution_strategy** = 0

The documentation for this struct was generated from the following file:

- process_settings.h

5.10 Stopwatch Class Reference

Public Member Functions

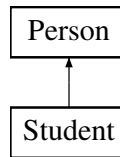
- **Stopwatch** (int _id, std::string _name)
- void **stopwatch_start** ()
- void **stopwatch_pause** ()
- long **stopwatch_stop** ()
- long **get_time_elapsed** () const
- long **clear_time_elapsed** ()
- std::string **get_name** () const
- int **get_id** () const

The documentation for this class was generated from the following files:

- timer.h
- timer.cpp

5.11 Student Class Reference

Inheritance diagram for Student:



Public Member Functions

- **Student** (std::string f_name="", std::string l_name="", std::vector< int > hw_scores={}, int exam_score=0)
- **Student** (const [Student](#) &other)
- [Student](#) & **operator=** (const [Student](#) &other)
- **Student** ([Student](#) &&other) noexcept
- [Student](#) & **operator=** ([Student](#) &&other) noexcept
- std::vector< int > **get_hw_scores** () const
- int **get_exam_score** () const
- double **get_final_score_avg** () const
- double **get_final_score_med** () const
- void **set_hw_scores** (std::vector< int > hw_scores)
- void **set_exam_score** (int exam_score)
- void **set_final_score_avg** (double final_score_avg)
- void **set_final_score_med** (double final_score_med)
- void **push_hw_score** (int hw_score)
- void **disallow_cval_mod** ()
- std::string **get_full_name** () override

Public Member Functions inherited from [Person](#)

- **Person** (std::string f_name, std::string l_name)
- std::string **get_f_name** () const
- std::string **get_l_name** () const
- void **set_f_name** (std::string f_name)
- void **set_l_name** (std::string l_name)

Friends

- std::ostream & **operator<<** (std::ostream &os, const [Student](#) &student)
- std::istream & **operator>>** (std::istream &is, [Student](#) &student)

Additional Inherited Members

Protected Attributes inherited from [Person](#)

- std::string **f_name**
- std::string **l_name**

5.11.1 Member Function Documentation

5.11.1.1 `get_full_name()`

```
std::string Student::get_full_name () [override], [virtual]
```

Implements [Person](#).

The documentation for this class was generated from the following files:

- `student.h`
- `student.cpp`

5.12 StudentTest Class Reference

Static Public Member Functions

- static bool **test_destructor** ()
- static bool **test_copy_constructor** ()
- static bool **test_copy_assignment_operator** ()
- static bool **test_move_constructor** ()
- static bool **test_move_assignment_operator** ()
- static std::tuple< bool, std::string > **test_all** ()
- static bool **test_input_operator** ()
- static bool **test_output_operator** ()

The documentation for this class was generated from the following files:

- `student_test.h`
- `student_test.cpp`

5.13 Timer Class Reference

Public Member Functions

- void **initialize_watch** (int watch_id, std::string watch_name)
- bool **start_watch** (int watch_id)
- bool **pause_watch** (int watch_id)
- bool **stop_watch** (int watch_id, std::string msg)
- long **get_time_elapsed** (int watch_id) const
- void **write_to_file** (const std::string &filename) const
- void **write_times** (const std::string &filename)

The documentation for this class was generated from the following files:

- `timer.h`
- `timer.cpp`

Chapter 6

File Documentation

6.1 cl_students.h

```
00001 #ifndef CL_STUDENTS_H
00002 #define CL_STUDENTS_H
00003
00004 #include <vector>
00005 #include <iostream>
00006 #include <limits>
00007
00008 #include "generator.h"
00009 #include "inputs.h"
00010 #include "student.h"
00011
00012 using std::cout;
00013 typedef std::numeric_limits<int> int_lim;
00014
00015 class CL_Students
00016 {
00017 public:
00018     static std::vector<Student> get_user_input(const bool &generate_names, const bool
&generate_grades);
00019     static void gen_names(Student &student);
00020     static void CL_names(Student &student, int students_size);
00021     static void gen_grades(Student &student, int students_size);
00022     static void CL_grades(Student &student, int students_size);
00023 };
00024
00025 #endif // CL_STUDENTS_H
```

6.2 constant_definitions.h

```
00001 #ifndef CONSTANT_DEFINITIONS_H
00002 #define CONSTANT_DEFINITIONS_H
00003
00004 #define NAME_LENGTH 17
00005 #define HW_WEIGHT 0.4
00006 #define EXAM_WEIGHT 0.6
00007 #define GRADE_MAX 10
00008 #define INPUT_FILE_NAME "kursiokai.txt"
00009
00010 #endif //CONSTANT_DEFINITIONS_H
```

6.3 file_gen.h

```
00001 #ifndef FILE_GEN_H
00002 #define FILE_GEN_H
00003
00004 #include <fstream>
00005 #include <iostream>
00006 #include <string>
```

```

00007 #include "generator.h"
00008 #include "constant_definitions.h"
00009 #include "timer.h"
00010
00011 class FileGen {
00012 public:
00013     static void gen_file(std::uint64_t num_students, std::uint64_t num_hw, const std::string
        &file_name);
00014 };
00015
00016
00017 #endif //FILE_GEN_H

```

6.4 generator.h

```

00001 #ifndef GENERATOR_H
00002 #define GENERATOR_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 class Generator {
00008     static const std::vector<std::string> first_names;
00009     static const std::vector<std::string> last_names;
00010
00011 public:
00012     static std::string gen_f_name() ;
00013     static std::string gen_l_name() ;
00014
00015     static std::string gen_name();
00016     static std::vector<std::string> gen_names(unsigned long num_of_name) const;
00017     static int rand_int(const int min, const int max);
00018
00019 };
00020
00021
00022 #endif //GENERATOR_H

```

6.5 inputs.h

```

00001 #ifndef INPUTS_H
00002 #define INPUTS_H
00003
00004 #include <iostream>
00005 #include <limits>
00006
00007 using std::cin;
00008 using std::cout;
00009 typedef std::numeric_limits<int> int_lim;
00010
00011 class CLInputs
00012 {
00013 public:
00014     static int numInput(const std::string &prompt, const int limit_max = int_lim::max(),
        const int limit_min = int_lim::min());
00015
00016     static std::string strInput(const std::string &prompt, const int limit_max = int_lim::max(),
        const int limit_min = int_lim::min());
00017 };
00018
00019
00020
00021 #endif // INPUTS_H

```

6.6 main.h

```

00001 #ifndef MAIN_H
00002 #define MAIN_H
00003
00004 #include "file_gen.h"
00005 #include "inputs.h"
00006 #include "read_students.h"
00007 #include "cl_students.h"
00008 #include "output_students.h"
00009 #include "students_calc.h"

```

```

00010 #include "process_settings.h"
00011 #include "student.h"
00012 #include "timer.h"
00013 #include "student_test.h"
00014 #include <limits>
00015 #include <tuple>
00016
00017 #endif // MAIN_H

```

6.7 output_students.h

```

00001 #ifndef OUTPUT_STUDENTS_H
00002 #define OUTPUT_STUDENTS_H
00003
00004 #include <fstream>
00005 #include <iomanip>
00006 #include <iostream>
00007 #include <vector>
00008
00009 #include "constant_definitions.h"
00010 #include "student.h"
00011
00012 struct Student;
00013 using std::cout;
00014
00015 class Output_students
00016 {
00017     std::ofstream output;
00018
00019 public:
00020     std::string error;
00021
00022     Output_students();
00023     ~Output_students();
00024
00025     void open_file(const std::string &file_name);
00026     void close_file();
00027     void output_students(std::vector<Student> const &students, const bool &out_to_file);
00028     void output_file(std::vector<Student> const &students);
00029     void output_terminal(std::vector<Student> const &students);
00030 };
00031
00032 #endif // OUTPUT_STUDENTS_H

```

6.8 person.h

```

00001 #ifndef PERSON_H
00002 #define PERSON_H
00003
00004 #include <string>
00005
00006 class Person
00007 {
00008 protected:
00009     std::string f_name, l_name;
00010
00011 public:
00012     Person(std::string f_name, std::string l_name) : f_name(f_name), l_name(l_name) {};
00013     Person() : f_name(""), l_name("") {};
00014     ~Person()
00015     {
00016         f_name.clear();
00017         l_name.clear();
00018     };
00019
00020     std::string get_f_name() const { return f_name; };
00021     std::string get_l_name() const { return l_name; };
00022
00023     void set_f_name(std::string f_name) { this->f_name = f_name; };
00024     void set_l_name(std::string l_name) { this->l_name = l_name; };
00025
00026     virtual std::string get_full_name() = 0;
00027 };
00028
00029 #endif // PERSON_H

```

6.9 process_settings.h

```

00001 #ifndef PROCESS_SETTINGS_H
00002 #define PROCESS_SETTINGS_H
00003
00004 struct Process_settings
00005 {
00006     bool generate_names = false;
00007     bool generate_grades = false;
00008     bool get_students_from_file = false;
00009     int sort_method = 0;
00010     bool output_to_file = false;
00011     bool generate_input_file = false;
00012     int input_file_student_num = 0;
00013     int input_file_student_hw = 0;
00014     std::string input_file_name;
00015     int distribution_strategy = 0;
00016 };
00017
00018 #endif // PROCESS_SETTINGS_H

```

6.10 read_students.h

```

00001 #ifndef READ_STUDENTS_H
00002 #define READ_STUDENTS_H
00003
00004 #include <fstream>
00005 #include <string>
00006 #include <vector>
00007 #include <sstream>
00008 #include <iostream>
00009 #include "student.h"
00010
00011 class File_students
00012 {
00013     int num_of_hw = 0;
00014     std::fstream file;
00015     std::string word;
00016
00017 public:
00018     int iteration = 0;
00019     std::string error;
00020     bool file_opened = false;
00021
00022     explicit File_students(const std::string &file_name);
00023     explicit File_students();
00024     ~File_students();
00025
00026     void open(const std::string &file_name);
00027     void openFile(const std::string &file_name);
00028     void closeFile();
00029     void count_hw();
00030     void read_students(std::vector<Student> &students);
00031 };
00032
00033 #endif // READ_STUDENTS_H

```

6.11 student.h

```

00001 #ifndef STUDENT_H
00002 #define STUDENT_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include "person.h"
00007 #include "constant_definitions.h"
00008 #include <vector>
00009 #include <string>
00010 #include <iostream>
00011 #include <limits>
00012
00013 class Student : public Person
00014 {
00015 private:
00016     std::vector<int> hw_scores;
00017     int exam_score{};
00018     double final_score_avg{};

```



```

00019     double final_score_med{};
00020     bool allow_cval_mod = true;
00021
00022 public:
00023     Student(std::string f_name = "", std::string l_name = "", std::vector<int> hw_scores = {}, int
exam_score = 0) : Person(f_name, l_name), hw_scores(hw_scores), exam_score(exam_score) {};
00024
00025     ~Student();
00026     Student(const Student &other);
00027
00028     Student &operator=(const Student &other);
00029     Student(Student &&other) noexcept;
00030     Student &operator=(Student &&other) noexcept;
00031
00032     friend std::ostream &operator<<(std::ostream &os, const Student &student);
00033     friend std::istream &operator>>(std::istream &is, Student &student);
00034
00035     std::vector<int> get_hw_scores() const;
00036     int get_exam_score() const;
00037     double get_final_score_avg() const;
00038     double get_final_score_med() const;
00039     void set_hw_scores(std::vector<int> hw_scores);
00040     void set_exam_score(int exam_score);
00041     void set_final_score_avg(double final_score_avg);
00042     void set_final_score_med(double final_score_med);
00043     void push_hw_score(int hw_score);
00044     void disallow_cval_mod();
00045     std::string get_full_name() override;
00046 };
00047
00048 #endif // STUDENT_H

```

6.12 student_test.h

```

00001 #ifndef STUDENT_TEST_H
00002 #define STUDENT_TEST_H
00003
00004 #include "student.h"
00005 #include "students_calc.h"
00006 #include <tuple>
00007 #include <string>
00008 #include <iostream>
00009 #include <sstream>
00010
00011 class StudentTest
00012 {
00013 public:
00014     static bool test_destructor();
00015     static bool test_copy_constructor();
00016     static bool test_copy_assignment_operator();
00017     static bool test_move_constructor();
00018     static bool test_move_assignment_operator();
00019     static std::tuple<bool, std::string> test_all();
00020     static bool test_input_operator();
00021     static bool test_output_operator();
00022 };
00023
00024 #endif // STUDENT_TEST_H

```

6.13 students_calc.h

```

00001 #ifndef STUDENTS_CALC_H
00002 #define STUDENTS_CALC_H
00003
00004 #include <vector>
00005 #include <algorithm>
00006
00007 #include "constant_definitions.h"
00008 #include "student.h"
00009
00010 class Calc_Students
00011 {
00012 public:
00013     static void calc_grades(std::vector<Student> &students);
00014     static void set_student_avg(Student &student);
00015     static void set_student_median(Student &student);
00016     static void sort_students(std::vector<Student> &students, int sort_method);
00017     static bool student_sort_f_name(Student const &lhs, Student const &rhs);

```

```

00018     static bool student_sort_l_name(Student const &lhs, Student const &rhs);
00019     static bool student_sort_avg(Student const &lhs, Student const &rhs);
00020     static bool student_sort_med(Student const &lhs, Student const &rhs);
00021     static double calc_average(std::vector<int> &&scores, int &&egzam);
00022 };
00023
00024 #endif // STUDENTS_CALC_H

```

6.14 timer.h

```

00001 #ifndef TIMER_H
00002 #define TIMER_H
00003
00004 #include <chrono>
00005 #include <string>
00006 #include <fstream>
00007 #include <vector>
00008
00009 using accuracy = std::chrono::milliseconds;
00010
00011 class Stopwatch
00012 {
00013     std::chrono::time_point<std::chrono::high_resolution_clock> start{};
00014     std::chrono::time_point<std::chrono::high_resolution_clock> end{};
00015     long time_elapsed = 0;
00016     bool stopped = false;
00017     std::string name;
00018     int id;
00019
00020 public:
00021     explicit Stopwatch(int _id, std::string _name);
00022
00023     void stopwatch_start();
00024     void stopwatch_pause();
00025     long stopwatch_stop();
00026     long get_time_elapsed() const;
00027     long clear_time_elapsed();
00028     std::string get_name() const { return name; }
00029
00030     int get_id() const;
00031 };
00032
00033 class Timer
00034 {
00035     std::string content;
00036     std::vector<Stopwatch> watches;
00037     bool times_written = false;
00038
00039 public:
00040     ~Timer();
00041
00042     void initialize_watch(int watch_id, std::string watch_name);
00043     bool start_watch(int watch_id);
00044     bool pause_watch(int watch_id);
00045     bool stop_watch(int watch_id, std::string msg);
00046     long get_time_elapsed(int watch_id) const;
00047     void write_to_file(const std::string &filename) const;
00048     void write_times(const std::string &filename);
00049 };
00050
00051 #endif // TIMER_H

```

Index

Calc_Students, [17](#)
CL_Students, [17](#)
CLInputs, [18](#)

File_students, [18](#)
FileGen, [18](#)

Generator, [19](#)
get_full_name
 Student, [22](#)

Output_students, [19](#)

Person, [19](#)
Process_settings, [20](#)

Release aprašymai, [1](#)

Stopwatch, [20](#)
Student, [21](#)
 get_full_name, [22](#)
StudentTest, [22](#)

Timer, [22](#)